

PYTHON

基礎

▶ NTNU CSIE CAMP

課堂須知

- 有問題多多提問或是使用slido發問
- 鼓勵多多提問



程式語言

辭書內容

名詞解釋：

用以傳達資訊的表示法、約定和規則的集合。語言由敘述組成，每個敘述則由單字或成語（詞）組成。組成時須有一定的語意，並遵照一定的語法規則。

資料來源：

國家教育研究院_語言

C reference

C89, C95, C99, C11, C17, C23 | Compiler support C99, C23

Language

- Basic concepts
- Keywords
- Preprocessor
- Expressions
- Declaration
- Initialization
- Functions
- Statements

Headers

Type support

Technical specifications

Dynamic memory exten

Floating-point extensio

Floating-point extensio

External Links – Non-ANSI

Program utilities

Variadic functions

Diagnostics library

Dynamic memory management

Strings library

Null-terminated strings

Date

Loca

Input

Algorithms library

Numerics library

Common mathematical functions

Floating-point environment (C99)

Pseudo-random number generation

Complex number arithmetic (C99)

Python »

English

3.13.5

3.13.5

3.13.5 Documentation » The Python Language Reference » 2. Lexical analysis

Theme

Auto

Quick search

Go

| previous | next | modules | index

Table of Contents

2. Lexical analysis

- 2.1. Line structure
 - 2.1.1. Logical lines
 - 2.1.2. Physical lines
 - 2.1.3. Comments
 - 2.1.4. Encoding declarations
 - 2.1.5. Explicit line joining
 - 2.1.6. Implicit line joining
 - 2.1.7. Blank lines
 - 2.1.8. Indentation
 - 2.1.9. Whitespace between tokens
- 2.2. Other tokens
- 2.3. Identifiers and keywords
 - 2.3.1. Keywords
 - 2.3.2. Soft Keywords
 - 2.3.3. Reserved classes of identifiers
- 2.4. Literals
 - 2.4.1. String and Bytes literals
 - 2.4.1.1. Escape sequences
 - 2.4.2. String literal concatenation
 - 2.4.3. f-strings
 - 2.4.4. Numeric literals
 - 2.4.5. Integer literals
 - 2.4.6. Floating-point literals
 - 2.4.7. Imaginary literals
- 2.5. Operators
- 2.6. Delimiters

2. Lexical analysis

A Python program is read by a *parser*. Input to the parser is a stream of [tokens](#), generated by the *lexical analyzer* (also known as the *tokenizer*). This chapter describes how the lexical analyzer breaks a file into tokens.

Python reads program text as Unicode code points; the encoding of a source file can be given by an encoding declaration and defaults to UTF-8, see [PEP 3120](#) for details. If the source file cannot be decoded, a [SyntaxError](#) is raised.

2.1. Line structure

A Python program is divided into a number of *logical lines*.

2.1.1. Logical lines

The end of a logical line is represented by the token `NEWLINE`. Statements cannot cross logical line boundaries except where `NEWLINE` is allowed by the syntax (e.g., between statements in compound statements). A logical line is constructed from one or more *physical lines* by following the explicit or implicit *line joining* rules.

2.1.2. Physical lines


```

        int k; //by Andy Sloane
        system("cls"); //C++ Builder
        float A= 0,B=0,i,j,z[1760];char
        b[1760];printf("\x1b[2J");for(;;
        ){memset(b,32,1760);memset(z,0,7040)
        ;for(j=0;6.28>j;j+=0.07)for(i=0;6.28
        >i;i+=0.02){float c=sin(i),d=cos(j),e=
        sin(A),f=sin(j),g=cos(A),h=d+2,D=1/(c*
        h*e+f*g+5),l=cos      (i),m=cos(B),n=s\
        in(B),t=c*h*g-f*      e;int x=40+30*D*
        (l*h*m-t*n),y=      12+15*D*(l*h*n
        +t*m),o=x+80*y,      N=8*((f*e-c*d*g
        )*m-c*d*e-f*g-l      *d*n);if(22>y&&
        y>0&&x>0&&80>x&&D>z[o]){z[o]=D;;;b[o]=
        ".,-~:;=!*#$@"[N>0?N:0];}} /*#####!!-*/
        printf("\x1b[H");for(k=0;1761>k;k++)
        putchar(k%80?b[k]:10);A+=0.04;B+=
        0.02;}/*****#####*****!!=;:~
        ~::==!!!*****!!!==:-
        .,~~;;;=====;~-.
        ..,-----,*/

```

程式語言 [編輯]

條目 討論 漢 漢 臺灣正體 ▼

維基百科，自由的百科全書



此條目需要補充更多來源。（2017年2月10日）

請協助補充多方面可靠來源以改善這篇條目，無法查證的內容可能會因為異議

致使用者：請搜尋一下條目的標題（來源搜尋：“程式語言” — 網頁、新聞、書

上是否存在該主題的更多可靠來源（判定指引）。

程式設計語言（英語：programming language），是用來定義電腦程式的**形式語言**。它是一種被**標準化**的交流技巧，用來向**電腦**發出指令，一種能夠讓**程式設計師**準確地定義電腦所需要使用資料的電腦語言，並精確地定義在不同情況下所應當採取的行動。

為甚麼要寫程式?

Visual Studio Code [編輯]

條目 討論 漢 漢 臺灣正體 ▾

維基百科，自由的百科全書



提示：此條目的主題不是 **Microsoft Visual Studio**。

Visual Studio Code（簡稱 **VS Code**）是一款由微軟開發且跨平台的免費原始碼編輯器^[7]。該軟體以擴充元件的方式支援語法突顯、程式碼自動補全（又稱 **IntelliSense**）、程式碼重構功能，並且內建了命令列工具和 **Git** 版本控制系統^[8]。使用者可以更改佈景主題和鍵盤捷徑實現個人化設定，也可以透過內建的擴充元件程式商店安裝其他擴充元件以加強軟體功能。



Visual Studio Code

<https://code.visualstudio.com> · [翻譯這個網頁](#) ·

Visual Studio Code - Code Editing. Redefined

Visual Studio Code redefines AI-powered coding with GitHub Copilot for building and debugging modern web and cloud applications. **Visual Studio Code** is free ...

[Download](#)

Visual Studio Code is free and available on your favorite ...

Docs

Find out how to set-up and get the most from Visual Studio Code ...

Windows

Use the Windows installer ... By default, VS Code is installed ...

Extensions

One place for all extensions for Visual Studio, Azure DevOps ...

VS Code for the Web

Visual Studio Code for the Web provides a free, zero-install ...



Visual Studio Code

軟體：

Visual Studio Code是一款由微軟開發且跨平台的免費原始碼編輯器。該軟體以擴充元件的方式支援語法突顯、程式碼自動補全、程式碼重構功能，並且內建了命令列工具和 **Git** 版本控制系統。使用... [More](#) ▾

Source: [維基百科](#) ▸

程式設計語言：[JavaScript](#)、[C語言](#)、[C#](#)、[CSS](#)、[TypeScript](#)

首次發行日期：2015 年 4 月 29 日

作業系統：[Windows](#) 7或更高版本、OS X 10.10或更高版本、Linux

原始碼庫：github.com/microsoft/vscode

平台：[IA-32](#)、[x86-64](#)、[ARM64](#)

授權條款：原始碼：[MIT授權條款](#)；可執行檔：免費軟體；

檔案大小：Windows：40.8–68.3 MB；Linux：46.5–66.6 MB；macOS：67.5 MB

尋找你的作業系統

 **Visual Studio Code**

[Docs](#) [Updates](#) [Blog](#) [API](#) [Extensions](#) [FAQ](#) [GitHub Copilot](#)

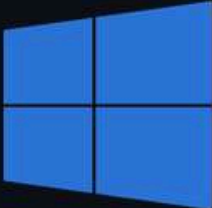


[Download](#)

 Get [GitHub Copilot Free](#) in VS Code! ×

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



 **Windows**
Windows 10, 11

User Installer [x64](#) [Arm64](#)

System Installer [x64](#) [Arm64](#)

.zip [x64](#) [Arm64](#)

CLI [x64](#) [Arm64](#)



 **.deb**
Debian, Ubuntu

 **.rpm**
Red Hat, Fedora, SUSE

.deb [x64](#) [Arm32](#) [Arm64](#)

.rpm [x64](#) [Arm32](#) [Arm64](#)

.tar.gz [x64](#) [Arm32](#) [Arm64](#)

Snap [Snap Store](#)

CLI [x64](#) [Arm32](#) [Arm64](#)



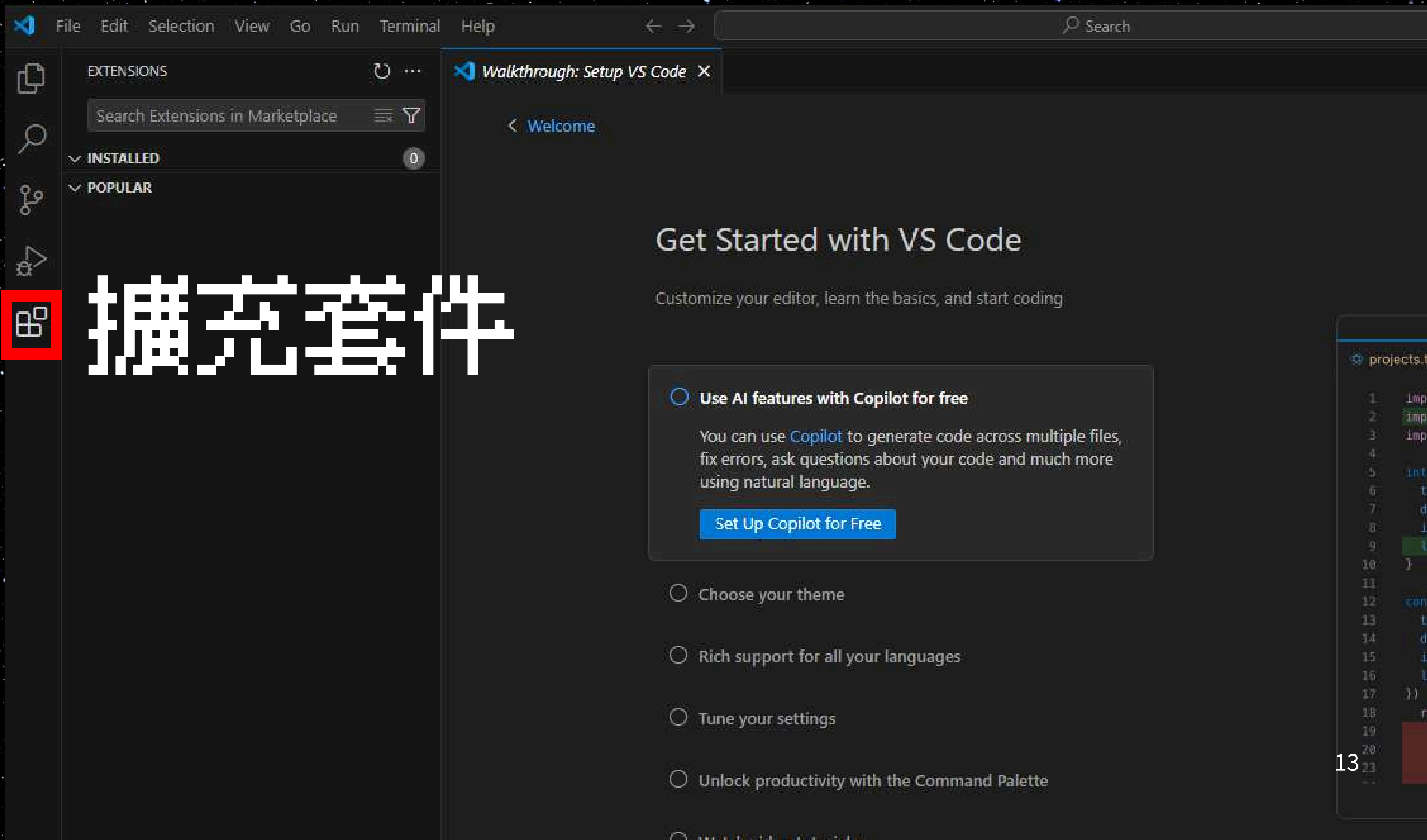
 **Mac**
macOS 10.15+

.zip [Intel chip](#) [Apple silicon](#) [Universal](#)

CLI [Intel chip](#) [Apple silicon](#)

按步驟安裝





擴充套件

Get Started with VS Code

Customize your editor, learn the basics, and start coding

☐ Use AI features with Copilot for free

You can use [Copilot](#) to generate code across multiple files, fix errors, ask questions about your code and much more using natural language.

[Set Up Copilot for Free](#)

☐ Choose your theme

☐ Rich support for all your languages

☐ Tune your settings

☐ Unlock productivity with the Command Palette

☐ Watch video tutorials

FileEditSelectionViewGoRunTerminalHelp

Search

EXTENSIONS: MARKETPLACE

python



Python

Python language support with exte...

Microsoft

Install

161.5M

4



Python Debugger

Python Debugger extension using d...

Microsoft

Install

71M

4.5



Python Indent

Correct Python indentation

Kevin Rose

Install

13.4M

4



Python Extension P...

Popular Visual Studio Code extensio...

Don Jayamanne

Install

10.9M

4.5



Python-for-VSCode

Python language extension for vsco...

Thomas Haakon Townse...

Install

5.9M

2



Python Environme...

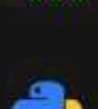
View and manage Python environm...

Don Jayamanne

Install

11.1M

3.5



autoDocstring - Pyt...

Generates python docstrings autom...

Nils Werner

Install

12.8M

5



Python-Preview

Provide Preview for Python Executio...

dongli

Install

2.2M

4.5



Python Extended

Python Extended is a vscode snippe...

Taiwo Kareem

Install

1.7M

4



Python Path

Python imports utils.

Mathias Gehrt

Install

1.4M

4

Extension: Python



Python

Microsoft [microsoft.com](#) | 161,544,358 | 5 stars (609)

Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger), linting, formatting, refactoring, u...

Install

Auto Update

DETAILS

FEATURES

CHANGE LOG

EXTENSION PACK

Python extension for Visual Studio Code

A Visual Studio Code extension with rich support for the Python language (for all actively supported Python versions), providing access points for extensions to seamlessly integrate and offer support for IntelliSense (Pylance), debugging (Python Debugger), formatting, linting, code navigation, refactoring, variable explorer, test explorer, and more!

Support for [vscode.dev](#)

The Python extension does offer some support when running on [vscode.dev](#) (which includes [github.dev](#)). This includes partial IntelliSense for open files in the editor.

Installed extensions

The Python extension will automatically install the following extensions by default to provide the best Python development experience in VS Code:

- [Pylance](#) - to provide performant Python language support
- [Python Debugger](#) - to provide a seamless debug experience with debugpy

These extensions are optional dependencies, meaning the Python extension will remain fully functional if they fail to be installed. A or all of these extensions can be disabled or uninstalled at the expense of some features. Extensions installed through the marketplace are subject to the [Marketplace Terms of Use](#).

Extensibility

Marketplace

Identifier	ms-python.python
Version	2025.2.0
Published	2016-01-19, 07:03:11
Last Released	2025-03-20, 03:51:18

Categories

Programming Languages

Debuggers

Data Science

Machine Learning

Resources

[Marketplace](#)

[Issues](#)

[Repository](#)

[License](#)

[Microsoft](#)

Would you like to change Visual Studio Code's display language to Chinese Traditional and restart?

Change Language and Restart

0 0 0

14

Google python

全部 影片 圖片 購物 新聞 網頁 地圖 更多 工具

Python.org
https://www.python.org · 翻譯這個網頁

Welcome to Python.org

Python is a programming language that lets you work quickly and integrate systems more effectively.
Learn More

Downloads Python 3.13.1 Python For Beginners The Python Tutorial

Python.org
https://www.python.org › downloads · 翻譯這個網頁

Download Python

Download the latest version of Python. Download Python 3.13.2. Looking for Python with a different OS? Python for Windows, Linux, macOS, Other.

Python 3.13.1 Python Releases for Windows Python Releases for macOS

相關問題：

Python自學要多久？

學Python可以做什麼工作？

Python 能做到什麼？

Python

程式語言：

Python 是一種廣泛使用的直譯式、進階和通用的程式語言。Python支援多種程式設計範式，包括結構化、程序式、反射式、物件導向和函數式程式設計。它擁有動態型別系統和垃圾回收功能，能...

Source: 維基百科

副檔名：.py、.pyi、.pyc、.pyd、.pyo (3.5之前)、.pyw、.pyz (自從3.5)

python™

Donate Search GO Socialize

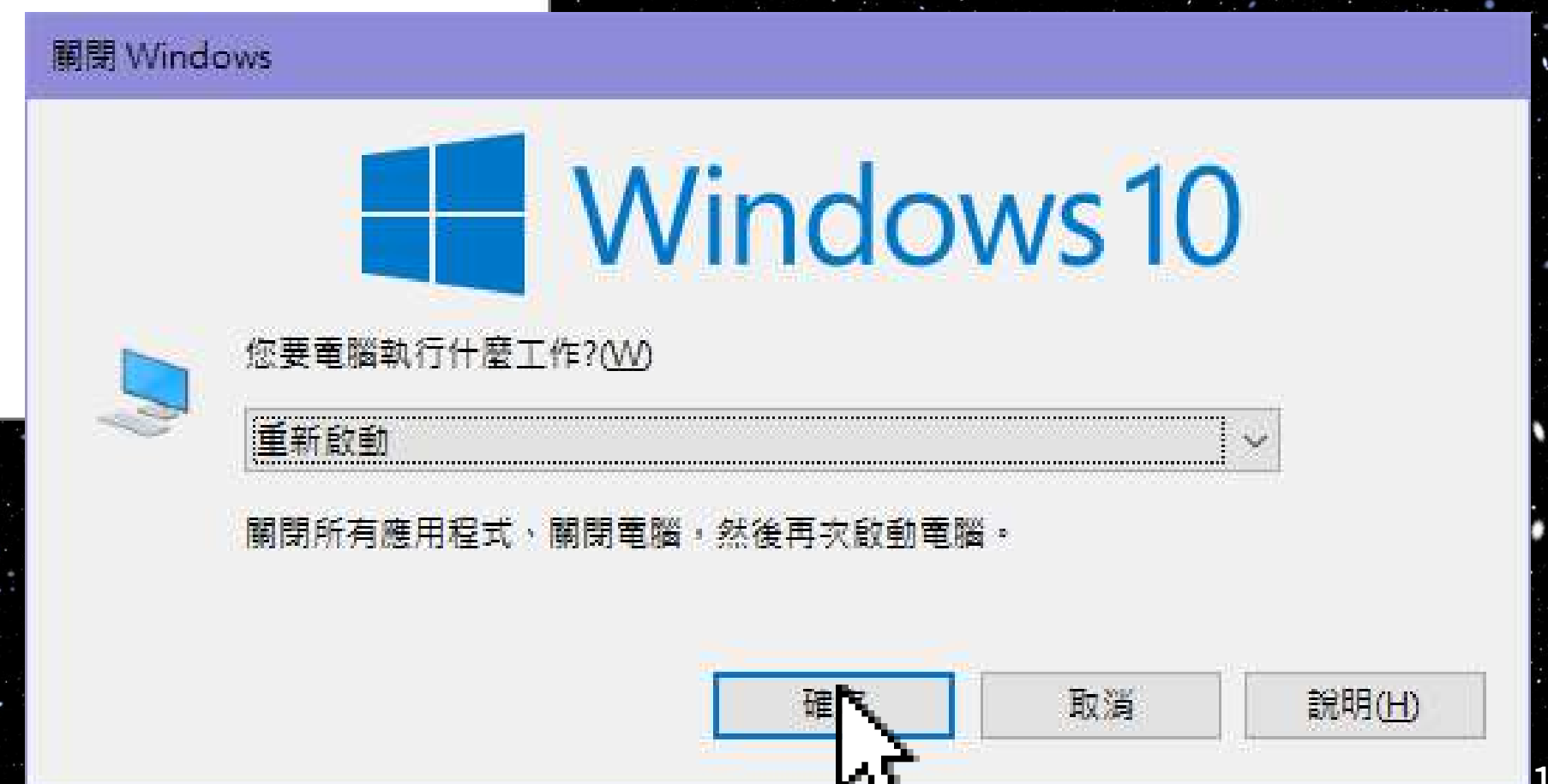
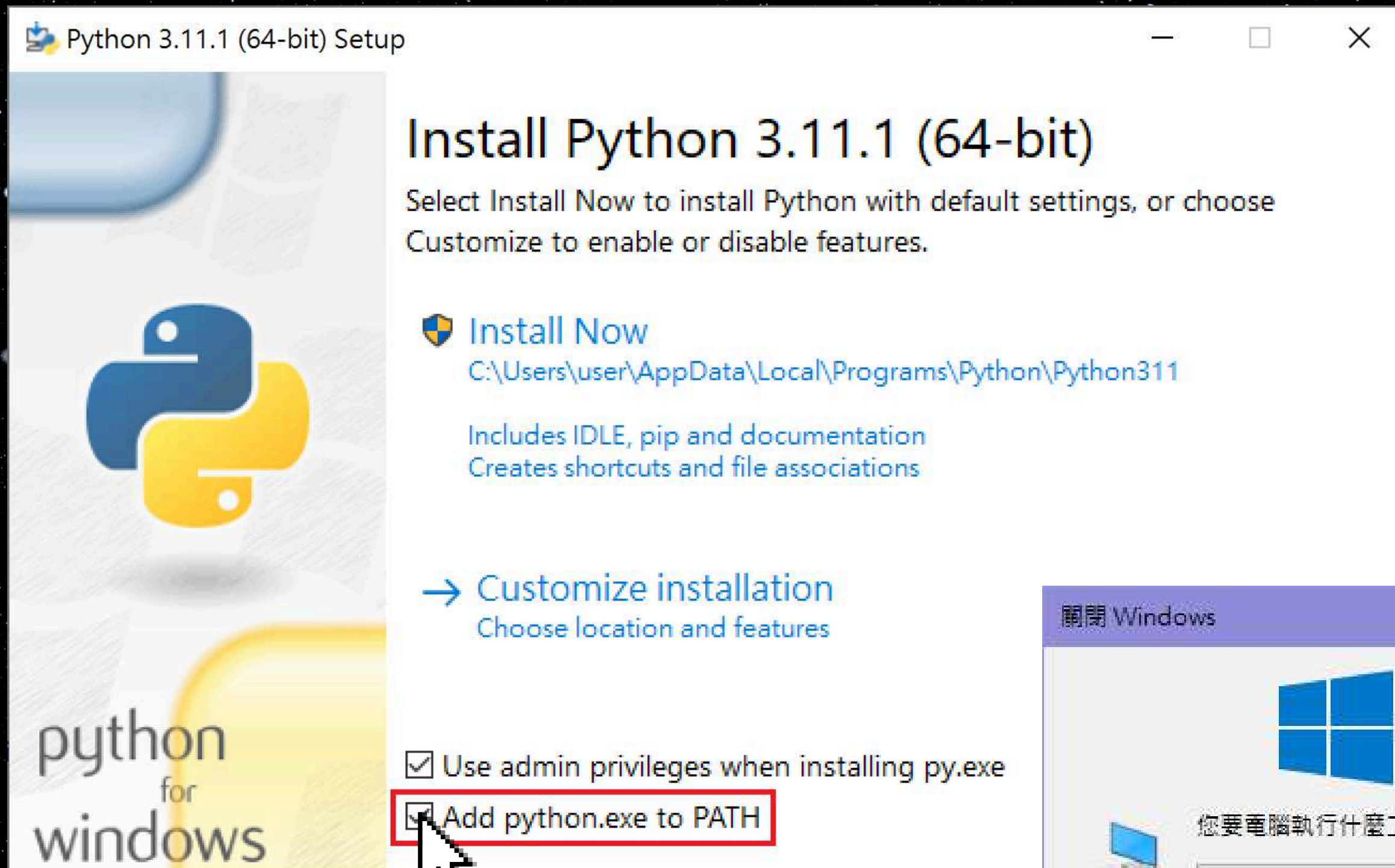
About Downloads Documentation Community Success Stories News Events

Download the latest version for Windows

Download Python 3.13.2

Looking for Python with a different OS? Python for [Windows](#), [Linux/UNIX](#), [macOS](#), [Other](#)

Want to help test development versions of Python 3.14? [Pre-releases](#), [Docker images](#)



Hello World

Hello World 程式碼由來

由來的部分根據[相關紀載](#)，我們要追溯回西元 1972 年，美國的科技研究機構貝爾實驗室成員布萊恩·柯林漢(Brian Wilson Kernighan)撰寫了一份內部檔案《A Tutorial Introduction to the Language B》，其中，它提到了 Hello World 的字串。

當時他使用 B 語言撰寫了第一個使用參數的 Hello World 相關程式：

```
main( ) {  
    extern a, b, c;  
    putchar(a); putchar(b); putchar(c); putchar('!\n');  
}  
  
a 'hell';  
b 'o, w';  
c 'orld';
```

而後在基於 B 語言創造 C 語言之後，就用更簡潔的方式來輸出 Hello World：

```
main( ) {  
    printf("hello, world\n");  
}
```

從此之後，**所有程式語言一開始輸出字串時**，都會以全部小寫的「hello, world」來當作第一個輸出的程式碼內容。

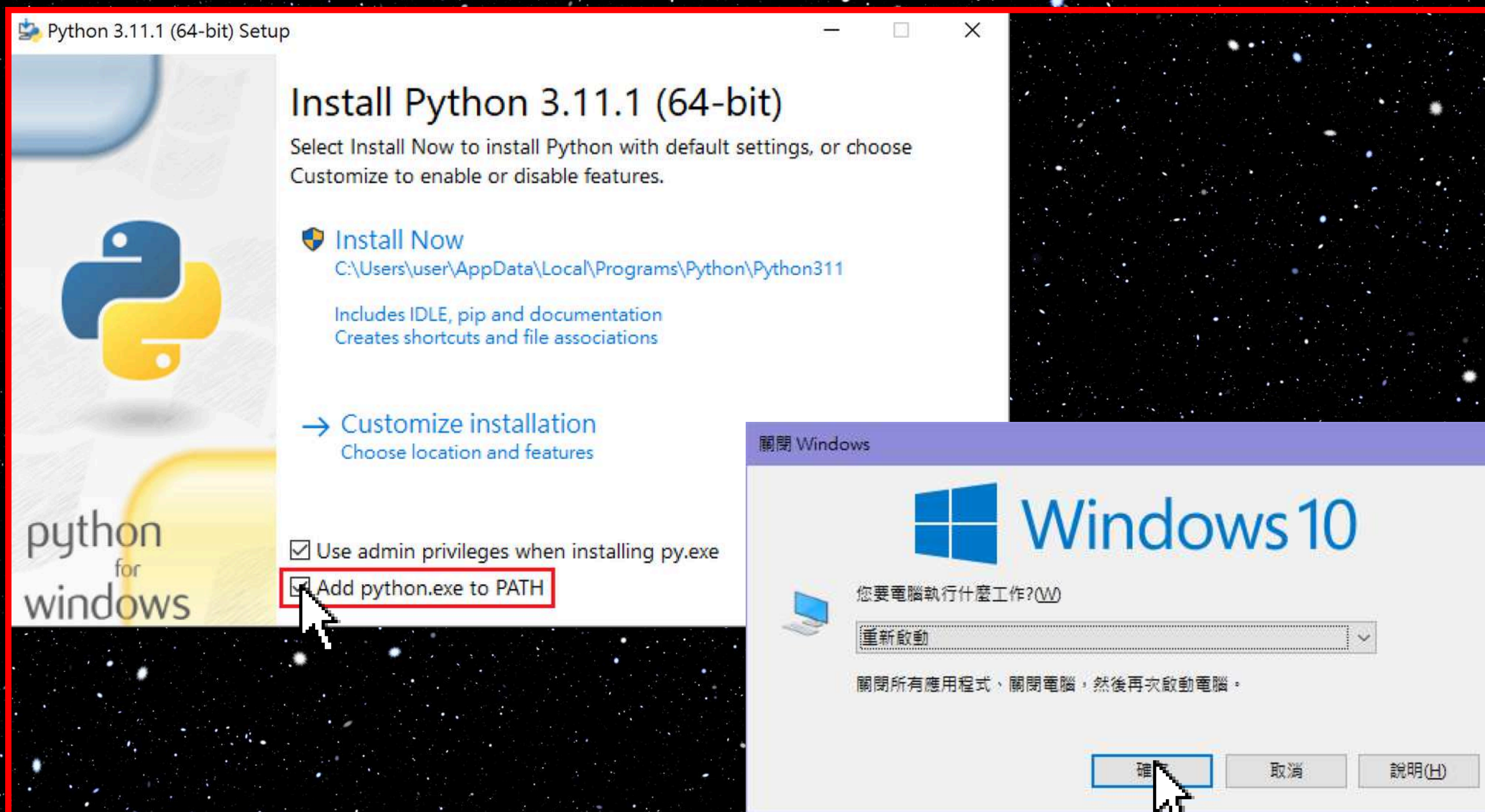
task (五分鐘)



```
print("Hello World")
```

`print()` 輸出在螢幕上

call back



Python [編輯]

條目 [討論](#) [漢](#) [漢](#) [臺灣正體](#) ▼

維基百科，自由的百科全書



關於與「**Python**」標題相近或相同的條目，請見「[Python \(消歧義\)](#)」。



此條目包含過多僅特定讀者會感興趣的**過度細節內容**。（2025年3月
請重新**整理**本條目以切合主題，並移除與**維基百科內容方針**相悖的過度細節內容。

Python（英語發音：/ˈpaɪθən/；英語發音：/ˈpaɪθɑːn/），是一種廣泛使用的**直譯式**、**進階**和**通用的程式語言**。Python支援多種程式設計範式，包括結構化、程式式、反射式、物件導向和函數式程式設計。它擁有**動態型別系統**和**垃圾回收**功能，能夠自動管理主記憶體使用，並且其本身擁有一個巨大而廣泛的標準庫。它的語言結構以及**物件導向**的方法，旨在幫助程式設計師為小型的和大型的專案編寫邏輯清晰的程式碼。

Python [編輯]

直譯語言 [編輯]

文A 24 種語言

條目 討論 漢 漢 臺灣正體

閱讀 編輯 檢視歷史 工具

維基百科，自由的百科全書



此條目沒有列出任何參考或來源。(2014年7月21日)

維基百科所有的內容都應該可供查證。請協助補充可靠來源以改善這篇條目。無法查證的內容可能會因為異議提出而被移除。

直譯語言（英語：Interpreted language）是一種**程式語言**類型。這種類型的程式語言，會將程式碼一句一句直接執行，不需要像**編譯語言**（Compiled language）一樣，經過**編譯器**先行編譯為**機器碼**，之後再執行。這種程式語言需要利用**直譯器**，在執行期，動態將程式碼逐句直譯（interpret）為機器碼，或是已經預先編譯為機器碼的**子程式**，之後再執行。

理論上，任何程式語言都可以是編譯式，或直譯式的。它們之間的區別，僅與程式的應用有關。許多程式語言同時採用編譯器與直譯器來實作，其中包括**Lisp**，**Pascal**，**BASIC** 與 **Python**。**JAVA**及**C#**採用混合方式，先將程式碼編譯為**位元組碼**，在執行時再進行直譯。

用，並且其本身擁有一個已入而廣泛的標準庫。它的語言結構以及物件等問的方法，旨在幫助程式設計師為小型的和大型的專案編寫邏輯清晰的程式碼。

comment (註解)

comment_example.py

快捷鍵

- 單行註解：
- Windows/Linux: Ctrl + /
- Mac: Cmd + /

variable (変数)

變數(Variables)

- 顧名思義, 是會隨著程式改變數值的數
- 改變變數數值的方式: 賦值
- Python 使用變數不需要宣告

```
x = 10
```


變數的命名規則

- 在Python中，變數名稱可以出現大小寫英文以及“_”
- 新版本的Python支援中文變數名稱
- 注意：但我強烈建議不要使用中文變數名稱
- 不要使用會變色的變數名稱

變數型態

1. 數值型態 (整數int、浮點數float、布林值bool)
2. 字串型態 (字串string)
3. 容器型態 (串列list、字典dict、元組tuple)

- 變數的型態可能會影響到程式的運作
- 可以使用 `type()` 來取得變數的型別

數值型態

- 整數 `integer` : 沒有小數點的數字
- 浮點數 `float` : 有小數點的數字
- 真假值(布林) `bool` : `True`、`False`

字串型態

- 字串(string)：一組文字資料，由「" "」 「' '」包含

變數轉型 (Casting)

- 有 `int()`、`float()`、`str()`……等
- 可以把 `float` 轉型成 `int`，取得浮點數的整數部分

輸入與輸出

- 輸出語法: `print("value1", "value2")`
- `sep`: 每個值之間的分隔符號
- `end`: 輸出結尾的符號

- 輸入語法: `input("input messages.")`
- `input`輸入的型態皆是“**字串**”，這意味著你無法直接利用輸入進行運算
- 這個時候就可以使用變數轉型

task (七分鐘)

輸入三個數字，並將三個數字輸出出來並以", "間隔

```
input x: 10
```

```
input y: 20
```

```
input z: 30
```

```
x: 10, y: 20, z: 30
```


陳述與運算式

陳述(statement)

- 陳述是程式執行的單位，程式的執行方式就是從上而下，一個陳述接著一個陳述執行的。



```
a = 10  
a = "32132"  
a = True  
a = 200
```

最後，a的值是200

幾個重要的陳述

- `if`, `else`, `elif` (判斷)
- `for`, `while`, `break`, `continue` (迴圈)
- `return` (函式回傳值)
- `import` (引入資料庫)

還有簡單但最重要的陳述，運算式陳述

運算式陳述、運算子與運算元

- 式子中進行運算的資料稱為運算元，連接運算元的符號就是運算子

3 + 2

運算元 運算子

- 由運算元與運算子構成的陳述式，就是運算式陳述
- 本課介紹的運算子有算術運算子、關係運算子、邏輯運算子

算數運算子

- 加法: $a+b$
- 減法: $a-b$
- 乘法: $a*b$
- 除法: a/b
- 除法取整數: $a // b$
- 除法取餘數: $a \% b$
- 指數: $a**b$

關係運算子

- A 大於 B: $A > B$
 - A 小於 B: $A < B$
 - A 大於等於 B: $A \geq B$
 - A 小於等於 B: $A \leq B$
 - A 等於 B: $A == B$
 - A 不等於 B: $A != B$
-
- 包含關係運算子的運算式, 運算結果的值就是布林值

邏輯運算子

- 連接布林運算元，然後得出一個新的布林值
- `and`: 當前後兩個布林值都是True，得出的值也是True
- `or`: 其中一個布林值為True，其值就是 True
- `not`: 顛倒原本真假值 (`not`只連接一個布林值)

task (七分鐘)

- 讓使用者輸入兩個數字
- 利用變數轉型輸出相加的結果

```
input x: 20  
input y: 30  
20 + 30 = 50
```

想一想：數字最大可以算到多大呢？

Numeric Types — int, float, complex

There are three distinct numeric types: *integers*, *floating-point numbers*, and *complex numbers*. In addition, Booleans are a subtype of integers. **Integers have unlimited precision**. Floating-point numbers are usually implemented using `double` in C; information about the precision and internal representation of floating-point numbers for the machine on which your program is running is available in `sys.float_info`. Complex numbers have a real and imaginary part, which are each a floating-point number. To extract these parts from a complex number `z`, use `z.real` and `z.imag`. (The standard library includes the additional numeric types `fractions.Fraction`, for rationals, and `decimal.Decimal`, for floating-point numbers with user-definable precision.)

判断式

If判斷式

語法:

```
if (statement):
```

```
    (code A)
```

```
else:
```

```
    (code B)
```

- 如果陳述為真, 執行下方的程式 (code A)
- 否則執行 (code B)

縮排

- 電腦會透過程式碼的縮排判斷該行程式碼被包含在哪個範圍裡面。

else, elif

語法:

```
If (statement1):
```

```
    (code A)
```

```
elif (statement2):
```

```
    (code B)
```

```
else:
```

```
    (code C)
```

- 當statement1為真的時候，就執行內部的程式(code A)
- elif: 如果statement1未滿足，而statement2滿足時，執行(code B)
- else: 當statement1 和 statement2 皆未滿足，執行(code C)

巢狀if

- If 函式中也可以包含很多個if 函式，我們稱之為巢狀 if 函式

```
if 玩家生命值 > 0:  
    if 攻擊按鈕被按下:  
        if 能量條足夠:  
            print("使用大招攻擊")  
        else:  
            print("使用普通攻擊")  
    else:  
        print("防禦中")  
else:  
    print("角色死亡")
```


別這麼做！

```
function register()
{
    if (!empty($_POST)) {
        $msg = '';
        if ($_POST['user_name']) {
            if ($_POST['user_password_new']) {
                if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {
                    if (strlen($_POST['user_password_new']) > 5) {
                        if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {
                            if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {
                                $user = read_user($_POST['user_name']);
                                if (!isset($user['user_name'])) {
                                    if ($_POST['user_email']) {
                                        if (strlen($_POST['user_email']) < 65) {
                                            if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {
                                                create_user();
                                                $_SESSION['msg'] = 'You are now registered so please login';
                                                header('Location: ' . $_SERVER['PHP_SELF']);
                                                exit();
                                            } else $msg = 'You must provide a valid email address';
                                        } else $msg = 'Email must be less than 64 characters';
                                    } else $msg = 'Email cannot be empty';
                                } else $msg = 'Username already exists';
                            } else $msg = 'Username must be only a-z, A-Z, 0-9';
                        } else $msg = 'Username must be between 2 and 64 characters';
                    } else $msg = 'Password must be at least 6 characters';
                } else $msg = 'Passwords do not match';
            } else $msg = 'Empty Password';
        } else $msg = 'Empty Username';
        $_SESSION['msg'] = $msg;
    }
    return register_form();
}
```



- 程式可讀性低，不易閱讀
- 程式出錯時難修改、維護

- 善用邏輯運算子
- 善用elif
- 更改方法


```
if 玩家生命值 > 0:  
    if 攻擊按鈕被按下:  
        if 能量條足夠:  
            print("使用大招攻擊")  
        else:  
            print("使用普通攻擊")  
    else:  
        print("防禦中")  
else:  
    print("角色死亡")
```

```
if 玩家生命值 <= 0:  
    print("角色死亡")  
elif 攻擊按鈕被按下 and 能量條足夠:  
    print("使用大招攻擊")  
elif 攻擊按鈕被按下:  
    print("使用普通攻擊")  
else:  
    print("防禦中")
```



task (十分鐘)

- 製作一個猜數字遊戲
- 設定一個數
- 讓玩家輸入一個數，告訴他太大或太小
- 如果答對了就輸出"答對了"

迴圈

迴圈 (Loop)

- 迴圈可以用來重複執行部分程式，省去編寫程式的時間與空間
- 可以分成for迴圈與while迴圈

Q: 輸出 1~5 的數
輸出 1~10 的數

main.py ×

```
1 print(1)
2 print(2)
3 print(3)
4 print(4)
5 print(5)
6 |
```

main.py ×

```
1 print(1)
2 print(2)
3 print(3)
4 print(4)
5 print(5)
6 print(6)
7 print(7)
8 print(8)
9 print(9)
10 print(10)
```


如果我要1~10000?

for迴圈 (for loop)

如果是用來計數的for 迴圈，可以這麼寫：

```
for 變數 in range(start, end, step):  
    <迴圈中的程式>
```

迴圈中，<迴圈中的程式>會重複執行，其中變數會從start逐次遞增step，直到變數值大於或等於end才停止迴圈 (當step > 0時)

- start 起始值，默認是0
- end 終止值
- step 遞增值，默認是1

For loop的特徵

- 有一個變數會隨著迴圈進行次數遞增
- 當變數超出範圍，迴圈就會停止
- 適合用在已知、有限次數的迴圈

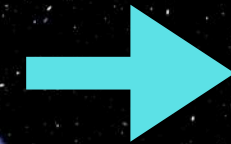
task (七分鐘)

- 製作一個猜數字遊戲
- 設定一個數
- 讓玩家輸入一個數，告訴他太大或太小
- 如果答對了就輸出"答對了"
- 讓玩家猜 10 次
- 提示玩家這是第幾次 or 告訴玩家剩幾次機會

巢狀迴圈

跟巢狀 `if` 的概念相似，迴圈內部也可以包含迴圈

```
for i in range(1,4,1):  
    for j in range(1,4,1):  
        (code A)  
        (code B)
```



AAAA B AAAA B AAAA B

- 九九乘法表

task (十二分鐘)

- 聖誕樹
- 輸入聖誕樹的高度
- 生成聖誕樹
- (一行要幾個空白?)
- (一行要幾個星星?)

```
聖誕樹的高度: 6
  *
 ***
*****
*****
*****
*****
*****
  |
```


while迴圈 (while loop)

語法:

```
while <關係運算的陳述 / 布林值>:  
    <迴圈內的程式>
```

進入迴圈時，會先確認上方陳述是否成立，之後每次輪迴也會確認陳述是否為真，反之則跳出迴圈

While loop的特徵

- 不知道迴圈重複的次數，但是知道迴圈停止的條件
- 如果滿足條件，就會無限次的執行下去
- 適合用在知道終止條件的迴圈，或是無限迴圈

迴圈控制

當迴圈在進行中，我們也可以透過條件判斷「跳出」迴圈

- `break`: 強制跳出整個迴圈，讓程式接續下去
- `continue`: 跳出這次迴圈，讓程式從下次迴圈開始

task (七分鐘)

- 製作一個猜數字遊戲
- 設定一個數
- 讓玩家輸入一個數，告訴他太大或太小
- 如果答對了就輸出"答對了"
- 讓玩家猜到對為止

串列

Q: 輸入5位同學的姓名、成績

```
1 name_one = "Andy"  
2 name_two = "Becky"  
3 name_three = "Charlie"  
4 name_four = "Danny"  
5 name_five = "Ethan"  
6 grade_one = 90  
7 grade_two = 80  
8 grade_three = 75  
9 grade_four = 105  
10 grade_five = 59  
11
```


如果今天有100個學生？

串列(List)

- 串列是一種容器型的資料型態，可以用來儲存一連串有順序性的資料

```
1 name = ["Andy", "Becky", "Charlie", "Danny", "Ethan"]  
2 grade = [90, 80, 75, 105, 59]  
3
```


值(value) / 索引值(index)

- 串列裡面的資料稱為值，代表該值的編號叫做索引值用，`name[4]` 可以取得 `name` 串列的第五個值
- 注意：`n` 個值的索引值是從第 0 個開始

index	0	1	2	3
value	0	97	95	75

```
myList = [0, 97, 95, 75]
```


task (七分鐘)

- 建立一個串列，並用程式將串列反轉
- Ex : [1, 2, 3, 4] $\longrightarrow$ [4, 3, 2, 1]
- 然後請不要用reverse()作弊，謝謝


```
int PyList_Append(PyObject *list, PyObject *item)
```

Part of the [Stable ABI](#).

Append the object *item* at the end of list *list*. Return 0 if successful; return -1 and set an exception if unsuccessful. Analogous to `list.append(item)`.

```
int PyList_Reverse(PyObject *list)
```

Part of the [Stable ABI](#).

Reverse the items of *list* in place. Return 0 on success, -1 on failure. This is the equivalent of `list.reverse()`.

[List Objects – Python 3.13.5 documentation](#)

二維串列

- 當一串列中的元素也是串列時，我們稱為二維串列



```
myList = [[1, 2, 3], [1, 2, 3], [1, 2, 3], [1, 2, 3]]
```


二維串列的應用

- 有時候一維串列無法方便的處理我們儲存資料的問題，這時候我們必須使用二維串列
- Ex:

1	2	3
1	2	3
1	2	3
1	2	3

task (二十分鐘)

- 圈圈叉叉
- 請玩家輸入他現在要下哪一格

```
...
...
...

輪到玩家1
第幾列： 2
第幾行： 2
=====

...
.X.
...
```


For in list?

For 迴圈不只可以拿來計數!

它也可以用來依次取得串列裡的元素做運算!

函式


```
1 a = 14
2
3 ▼ if a%2!=0 and a%3!=0 and a%5!=0 and a%7!=0:
4     print("是質數")
5 ▼ else:
6     print("不是質數")
7
8 a = a - 11
9
10 ▼ if a%2!=0 and a%3!=0 and a%5!=0 and a%7!=0:
11     print("是質數")
12 ▼ else:
13     print("不是質數")
14
15 a = a * 5
16
17 ▼ if a%2!=0 and a%3!=0 and a%5!=0 and a%7!=0:
18     print("是質數")
19 ▼ else:
20     print("不是質數")
```



都在做重複的事情

函式(function)

函式是一個建構程式的一個個區塊，只要在程式中呼叫函式，電腦就會自動執行函式內部的程式。

函式的結構

先來講沒有回傳值的函式：

```
● ● ● def: 定義(define)  
def printlist(mylist): arg: 參數(argument)  
    for i in mylist:  
        print(i, end = " ")  
    print()
```


參數的個數

- 一個函式中參數的個數由定義決定
- 每個參數用", "相隔
- 呼叫函式的時候需要代入相對應個數的參數

回傳值

有回傳值的函式和變數的概念較為相似，可以把牠當成是經過函式內部程式運算過後的變數。

回傳數值過後的函式便會停止。

用函式的好處

- 可以重複使用某一段程式碼，節省空間和時間
- 如果函式內的程式有問題，可以一次修改同樣的程式
- 可以增加程式的可讀性，使程式變得更簡潔
- 封裝：知其然不知其所以然

匯入外部函式

`import` (模組名稱)

以`random`來舉例

task (五分鐘)

- 製作一個猜數字遊戲
- 設定一個隨機數
- 讓玩家輸入一個數，告訴他太大或太小
- 如果答對了就輸出"答對了"
- 讓玩家猜到對為止

綜合實作題

task (三十分鐘)

二十一點